

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Facultad de Ingeniería

ARQUITECTURAS DE SOFTWARE: FUNDAMENTOS TEÓRICOS, ESTILOS, PATRONES Y CRITERIOS DE SELECCIÓN

Estudiantes / Autores:

Sebastián Avendaño Rodríguez
Cód. 20232020101

Juan Sebastián Rodríguez Serrano
Cód. 20232020038

Manuel Arturo Rueda Guiza
Cód. 20232020052

Asignatura:

Fundamentos de Ingeniería de
Software

Programa Académico:

Ingeniería de Sistemas

Bogotá D.C., Colombia

14 de septiembre de 2026

Índice

1. Fundamentos Teóricos de la Arquitectura de Software	3
1.1. Definición formal y alcance	3
1.2. Diferenciación conceptual: Arquitectura, Diseño y Estructura	3
1.3. Elementos primitivos de una arquitectura	3
1.4. Métricas analíticas de modularidad: Acoplamiento e Inestabilidad	4
1.5. Atributos de calidad fundamentales (ISO/IEC 25010 y SEI)	4
2. Análisis Sintético de Estilos y Patrones Arquitectónicos	5
2.1. Arquitectura Monolítica	5
2.2. Arquitectura en Capas (Layered Architecture)	5
2.3. Arquitectura Cliente-Servidor	5
2.4. Arquitectura MVC (Model-View-Controller)	6
2.5. Arquitectura Basada en Componentes (CBD)	6
2.6. Arquitectura Orientada a Servicios (SOA)	7
2.7. Arquitectura de Microservicios	7
2.8. Arquitectura Orientada a Eventos (Event-Driven Architecture - EDA)	7
2.9. Arquitectura Hexagonal (Ports and Adapters)	8
2.10. Arquitectura Clean (Clean Architecture)	8
2.11. Arquitectura Basada en Microkernel (Plug-in Architecture)	9
3. Principios, Patrones y Mecanismos de Implementación	9
3.1. Taxonomía conceptual	9
3.2. Principios fundamentales de diseño	9
3.3. Patrones de diseño estructurales aplicados	10
3.4. Mecanismos de implementación y persistencia	10
4. Comunicación entre Componentes y Persistencia de Datos	11
4.1. Mecanismos de comunicación inter-componente	11
4.2. Arquitectura de datos y consistencia distribuida	11

5. Comparación de Arquitecturas de Software	12
6. Caso Práctico: Plataforma de Comercio Electrónico	12
6.1. Planteamiento del problema	12
6.2. Evaluación arquitectónica comparada	12
6.3. Selección justificada de la solución técnica	13
7. Guía de Selección y Síntesis Técnica	13

1. Fundamentos Teóricos de la Arquitectura de Software

1.1. Definición formal y alcance

Según el estándar ISO/IEC/IEEE 42010 (2011), la arquitectura de software se define como la organización fundamental de un sistema encarnada en sus componentes, las relaciones entre ellos y con su entorno, y los principios que guían su diseño y evolución. Complementariamente, Bass, Clements y Kazman (2021) establecen que la arquitectura abarca el conjunto de estructuras requeridas para razonar sobre el sistema, enfocándose en las decisiones de diseño estructurales y de comportamiento de mayor costo de cambio, indispensables para asegurar los atributos de calidad.

1.2. Diferenciación conceptual: Arquitectura, Diseño y Estructura

- **Arquitectura:** Estrategia estructural de alto nivel, partición en subsistemas, selección de estilos y garantía de requisitos no funcionales globales. Posee el costo de reversión más elevado.
- **Diseño:** Táctica interna de componentes; definición de clases, métodos, algoritmos y patrones de diseño (e.g., GoF) dentro de las fronteras fijadas por la arquitectura.
- **Estructura:** Disposición física y topológica del código fuente, jerarquía de paquetes, enlaces binarios y organización del espacio de memoria en tiempo de ejecución.

1.3. Elementos primitivos de una arquitectura

Toda arquitectura formaliza tres entidades primarias (Shaw & Garlan, 1996):

1. **Componentes:** Unidades computacionales y almacenes de datos encapsulados (servicios, clientes, almacenes persistentes).
2. **Conectores:** Mecanismos mediadores de interacción, sincronización y transferencia de datos (invocaciones a métodos, tuberías, canales gRPC, brokers).
3. **Topología:** Grafo de configuración que restringe la geometría y los canales de comunicación admisibles entre componentes y conectores.

1.4. Métricas analíticas de modularidad: Acoplamiento e Inestabilidad

- **Acoplamiento Eferente (C_e):** Dependencias de salida hacia componentes externos.
- **Acoplamiento Aferente (C_a):** Dependencias de entrada provenientes de componentes externos.
- **Métrica de Inestabilidad (I):** Formulada por Martin (2003) para evaluar la resistencia al cambio de un módulo:

$$I = \frac{C_e}{C_a + C_e} \quad \text{donde } I \in [0, 1] \quad (1)$$

Un valor $I = 0$ caracteriza un componente de máxima estabilidad (difícil de alterar sin afectar a terceros); $I = 1$ define un módulo inestable y flexible.

- **Falta de Cohesión en Métodos (LCOM):** Cuantifica la disociación entre los métodos de una clase y sus variables de instancia, indicando si una clase viola el principio de responsabilidad única (Chidamber & Kemerer, 1994).

1.5. Atributos de calidad fundamentales (ISO/IEC 25010 y SEI)

- **Rendimiento:** Latencia, tiempo de respuesta y tasa de transferencia (*throughput*) bajo cargas definidas.
- **Escalabilidad:** Capacidad de absorber incrementos de carga mediante escalamiento vertical u horizontal.
- **Disponibilidad:** Proporción de tiempo operacional calculada a través del Tiempo Medio Entre Fallos (*MTBF*) y el Tiempo Medio de Recuperación (*MTTR*):

$$\text{Disponibilidad} = \frac{MTBF}{MTBF + MTTR} \quad (2)$$

- **Seguridad:** Preservación de confidencialidad, integridad, autenticación y no repudio.
- **Mantenibilidad, Fiabilidad y Testabilidad:** Grado de facilidad para aislar defectos, mantener continuidad operativa y verificar el comportamiento mediante pruebas reproducibles.

2. Análisis Sintético de Estilos y Patrones Arquitectónicos

2.1. Arquitectura Monolítica

- **Concepto y Estructura:** Todas las capacidades funcionales (presentación, reglas de dominio y persistencia) se compilan, empaquetan y ejecutan en un único proceso sobre el sistema operativo, compartiendo un único esquema de base de datos relacional.
- **Mecanismo Operativo:** Las invocaciones intermodulares ocurren en memoria compartida mediante llamadas a funciones nativas, beneficiándose de transacciones locales ACID homogéneas.
- **Ventajas y Desventajas:** Presenta máxima simplicidad de despliegue inicial y latencia de comunicación despreciable; no obstante, impone un acoplamiento temporal estricto, largos tiempos de compilación y tiende a la degradación arquitectónica (*Big Ball of Mud*).
- **Caso de Uso Óptimo:** Productos Mínimos Viables (MVP), aplicaciones de baja complejidad o equipos de desarrollo pequeños (≤ 8 ingenieros).

2.2. Arquitectura en Capas (Layered Architecture)

- **Concepto y Estructura:** Divide el sistema horizontalmente en estratos jerárquicos ordenados: Presentación, Negocio, Persistencia y Base de Datos (Richards & Ford, 2020).
- **Mecanismo Operativo:** En capas *cerradas*, una capa únicamente puede invocar a su capa inferior adyacente, garantizando el aislamiento de capas (*layers of isolation*). En capas *abiertas*, se omiten niveles para evitar sobrecostes, arriesgando el acoplamiento.
- **Ventajas y Desventajas:** Clara separación de responsabilidades y facilidad de modelado; sin embargo, induce al antipatrón *Sinkhole* (flujo de datos pasante sin lógica) y promueve el acoplamiento directo a los esquemas de bases de datos.
- **Caso de Uso Óptimo:** Aplicaciones corporativas estándar centradas en flujos transaccionales CRUD.

2.3. Arquitectura Cliente-Servidor

- **Concepto y Estructura:** Particiona las cargas de trabajo entre proveedores de recursos centralizados (*servidores*) y demandantes de servicios (*clientes*), operando sobre protocolos de red estandarizados (HTTP, TCP, gRPC).

- **Mecanismo Operativo:** Se divide en clientes ligeros (la lógica de presentación reside en el cliente; el procesamiento en el servidor) y clientes pesados (la computación y persistencia temporal residen en la estación local). Implementa esquemas de 2-Tier, 3-Tier y N-Tier.
- **Ventajas y Desventajas:** Centraliza el control de acceso y las políticas de seguridad; en contraste, el servidor se transforma en un punto único de fallo (SPOF) y la red introduce latencias variables.
- **Caso de Uso Óptimo:** Servicios financieros centrales consumidos por aplicaciones de banca móvil o terminales de escritorio.

2.4. Arquitectura MVC (Model-View-Controller)

- **Concepto y Estructura:** Patrón centrado en la interacción donde el **Modelo** encapsula el estado y las reglas de dominio, la **Vista** gestiona la representación visual y el **Controlador** traduce eventos de usuario en mutaciones sobre el modelo (Buschmann et al., 1996).
- **Mecanismo Operativo:** En su variante web pasiva, el controlador recibe la petición HTTP, actualiza el modelo y despacha la vista adecuada. No debe confundirse con una arquitectura de backend completa; suele operar dentro de la capa de presentación.
- **Ventajas y Desventajas:** Facilita vistas múltiples sobre los mismos datos y desacopla el diseño gráfico del modelo; su riesgo radica en la hipertrofia del controlador (*Fat Controller*).
- **Caso de Uso Óptimo:** Interfaces web de usuario enriquecidas y aplicaciones cliente interactivas.

2.5. Arquitectura Basada en Componentes (CBD)

- **Concepto y Estructura:** Descompone el sistema en piezas independientes, autónomas y sustituibles denominadas componentes, delimitadas por contratos estrictos (Szyperski, 2002).
- **Mecanismo Operativo:** Cada componente expone *interfaces provistas* (servicios ofrecidos) e *interfaces requeridas* (dependencias del entorno), comportándose como cajas negras donde la implementación queda oculta.
- **Ventajas y Desventajas:** Alta reutilización y sustitución modular; desventajas asociadas a la gestión de compatibilidad de versiones y conflictos de dependencias transversales.
- **Caso de Uso Óptimo:** Entornos modulares con empaquetado dinámico como OSGi o sistemas de librerías empresariales distribuidas.

2.6. Arquitectura Orientada a Servicios (SOA)

- **Concepto y Estructura:** Enfoque de integración empresarial donde capacidades de negocio heterogéneas se publican como servicios de granularidad gruesa coordinados por un Bus de Servicios Empresarial (*Enterprise Service Bus - ESB*) (Erl, 2005).
- **Mecanismo Operativo:** El ESB centraliza el enrutamiento inteligente, la transformación de protocolos (e.g., SOAP, XML, JMS) y la orquestación de procesos de negocio (BPEL).
- **Ventajas y Desventajas:** Facilita la interoperabilidad y reutilización de sistemas legados corporativos y mainframes; no obstante, el ESB se vuelve un cuello de botella sistémico complejo y costoso de mantener.
- **Caso de Uso Óptimo:** Integración a escala empresarial de plataformas institucionales dispares.

2.7. Arquitectura de Microservicios

- **Concepto y Estructura:** Estructura la aplicación como un conjunto de servicios autónomos, de granularidad fina, modelados alrededor de un contexto delimitado (*Bounded Context*) y desplegados de manera independiente (Newman, 2021).
- **Mecanismo Operativo:** Aplica el principio de «extremos inteligentes y tuberías tontas» mediante APIs REST, gRPC y brokers. Exige de forma obligatoria **Database-per-Service**, eliminando el acoplamiento a nivel de persistencia. Se complementa con componentes de soporte: API Gateways, Service Discovery y Service Meshes.
- **Ventajas y Desventajas:** Permite escalabilidad elástica independiente y autonomía de equipos multidisciplinares; sus penalizaciones son la alta complejidad operacional, observabilidad distribuida compleja y la pérdida de consistencia transaccional inmediata.
- **Caso de Uso Óptimo:** Sistemas de alta escala con dominios de negocio complejos gestionados por múltiples equipos paralelos.

2.8. Arquitectura Orientada a Eventos (Event-Driven Architecture - EDA)

- **Concepto y Estructura:** Estilo distribuido y asíncrono donde la computación responde a la emisión, captura y reacción ante **eventos de negocio** inmutables (Richards & Ford, 2020).
- **Mecanismo Operativo:** Los productores emiten eventos a canales de mensajería (Kafka, RabbitMQ) sin conocer a los receptores. Opera bajo topología de *Mediador* (orquestación)

o *Broker* (coreografía pura). Puede apoyarse en *Event Sourcing* y transferencia de estado mediante eventos (ECST).

- **Ventajas y Desventajas:** Máximo desacoplamiento espacial y temporal, alta reactividad y absorción de picos transaccionales; dificulta la consistencia global, la trazabilidad de flujos y la idempotencia de consumidores.
- **Caso de Uso Óptimo:** Procesamiento de flujos de datos en tiempo real, telemetría IoT y sistemas reactivos de alta concurrencia.

2.9. Arquitectura Hexagonal (Ports and Adapters)

- **Concepto y Estructura:** Formulada por Alistair Cockburn (2005) para aislar la lógica de dominio de las herramientas de infraestructura externa, bases de datos e interfaces de usuario.
- **Mecanismo Operativo:** El dominio se ubica en el centro. Expone **puertos primarios** (interfaces consumidas por la entrada: REST, CLI) y define **puertos secundarios** (interfaces requeridas para la salida: persistencia, mensajería). Los **adaptadores** traducen entre el mundo exterior y los puertos abstractos.
- **Ventajas y Desventajas:** Testabilidad aislada total del dominio sin infraestructura; neutralidad tecnológica absoluta frente a frameworks; penalizada por la sobrecarga de clases adaptadoras y mapeos constantes de datos.
- **Caso de Uso Óptimo:** Software con lógica de negocio compleja sujeta a evolución tecnológica continua.

2.10. Arquitectura Clean (Clean Architecture)

- **Concepto y Estructura:** Síntesis formal de Robert C. Martin (2017) que estructura el software en capas concéntricas: Entidades (reglas de empresa), Casos de Uso (reglas de aplicación), Adaptadores de Interfaz (controladores, presentadores) y Frameworks/Drivers (BD, Web).
- **Mecanismo Operativo:** Se rige por la **Regla de Dependencia**: *las dependencias de código fuente solo pueden apuntar hacia adentro*. El dominio desconoce la existencia de la base de datos o frameworks. El cruce de fronteras se realiza mediante Inversión de Dependencias (DIP).
- **Ventajas y Desventajas:** Máxima mantenibilidad y desacoplamiento de frameworks; induce fricción inicial por proliferación de capas de abstracción y conversiones de estructuras de datos (DTOs).

- **Caso de Uso Óptimo:** Sistemas corporativos de larga vida útil donde las reglas de negocio deben preservarse intactas ante cambios de infraestructura.

2.11. Arquitectura Basada en Microkernel (Plug-in Architecture)

- **Concepto y Estructura:** Separa un **Sistema Central** (*Core*) mínimo e indispensable de múltiples extensiones modulares autónomas denominadas **plugins** (Richards & Ford, 2020).
- **Mecanismo Operativo:** El Core expone un registro de plugins y los contratos de interfaz base. Los plugins extienden las funciones del sistema acoplándose exclusivamente a las especificaciones del núcleo, pudiendo cargarse de forma dinámica o estática.
- **Ventajas y Desventajas:** Excelente extensibilidad y aislamiento modular; diseñar una API de Core que permanezca estable ante extensiones imprevistas reviste gran complejidad.
- **Caso de Uso Óptimo:** Entornos de desarrollo integrados (e.g., Eclipse, VS Code), motores de cálculo o herramientas de diseño extensibles.

3. Principios, Patrones y Mecanismos de Implementación

Para prevenir ambigüedades técnicas, se clasifica rigurosamente el andamiaje conceptual de la ingeniería de software en tres niveles disyuntos:

3.1. Taxonomía conceptual

- **Principios de Diseño:** Directrices axiológicas universales de carácter no prescriptivo que rigen la modularidad y el desacoplamiento (e.g., SOLID, SoC, DRY, KISS, YAGNI).
- **Patrones de Diseño:** Soluciones estándar a problemas recurrentes de diseño de clases, objetos e interfaces a nivel microestructural (e.g., Facade, Adapter).
- **Mecanismos de Implementación:** Técnicas de programación y patrones de persistencia táctica que facilitan la operatividad de los principios (e.g., Inyección de Dependencias, Repository, Unit of Work).

3.2. Principios fundamentales de diseño

- **SOLID (Martin, 2003):**
 1. *Single Responsibility (SRP)*: Un módulo debe tener una única razón para cambiar.

2. *Open/Closed (OCP)*: Abierto a la extensión, cerrado a la modificación.
 3. *Liskov Substitution (LSP)*: Los subtipos deben poder sustituir a sus tipos base sin alterar la corrección del programa.
 4. *Interface Segregation (ISP)*: Múltiples interfaces específicas son preferibles a una interfaz de propósito general.
 5. *Dependency Inversion (DIP)*: Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.
- **Separation of Concerns (SoC)**: División del software en secciones exclusivas para cada responsabilidad funcional o técnica (Dijkstra, 1982).
 - **DRY / KISS / YAGNI**: Eliminación de duplicación de conocimiento (DRY), preservación de la simplicidad de diseño (KISS) y restricción de desarrollo a requerimientos explícitos actuales (YAGNI).

3.3. Patrones de diseño estructurales aplicados

- **Facade (Fachada)**: Interfaz simplificada que enmascara la complejidad interna de un subsistema complejo (Gamma et al., 1994).
- **Adapter (Adaptador)**: Transforma la interfaz de una clase en otra esperada por el cliente, permitiendo la cooperación de componentes incompatibles; constituye la base operativa de los adaptadores en Arquitectura Hexagonal.

3.4. Mecanismos de implementación y persistencia

- **Inversión de Control (IoC) e Inyección de Dependencias (DI)**: IoC invierte el flujo de control tradicional. DI materializa IoC extrayendo la creación de dependencias fuera del consumidor, inyectándolas en tiempo de inicialización (comúnmente vía constructor).
- **Repository Pattern**: Abstrae el acceso a los datos emulando una colección de objetos en memoria, aislando a la capa de dominio de sentencias de consulta o mapeadores ORM (Evans, 2004).
- **Unit of Work**: Registra las mutaciones ocurridas sobre agregados dentro de una transacción de negocio y coordina la escritura atómica de cambios en el almacén de datos (Fowler, 2002).

4. Comunicación entre Componentes y Persistencia de Datos

4.1. Mecanismos de comunicación inter-componente

- **REST (Fielding, 2000):** Síncrono sobre HTTP/JSON. Ubicuo, con soporte nativo de caché, pero afectado por latencias de texto y problemas de *over/under-fetching*.
- **GraphQL:** Síncrono. Permite al cliente especificar exactamente el grafo de atributos demandado, mitigando el sobre-consumo a costa de complejidad en caché de red.
- **gRPC:** Síncrono binario sobre HTTP/2 mediante Protocol Buffers. Garantiza tipado estricto en tiempo de compilación y latencias mínimas; ideal para tráfico interno (*east-west*) entre servicios.
- **Sistemas de Mensajería Asíncrona:**
 - *Colas Punto a Punto (AMQP):* Balanceo de carga donde un único consumidor procesa cada mensaje.
 - *Publish/Subscribe (Kafka):* Tópicos con retención de eventos consumidos de forma independiente por múltiples suscriptores; garantiza desacoplamiento temporal total.

4.2. Arquitectura de datos y consistencia distribuida

- **SQL vs. NoSQL:** Las bases relacionales (ACID) resuelven integridad estricta en dominios altamente conectados; los esquemas NoSQL (documentales, clave-valor, columnares) ofrecen escalamiento elástico y modelos desnormalizados.
- **Teorema CAP y PACELC:** Brewer (2000) demostró que ante una partición de red (P), un sistema distribuido solo puede garantizar Consistencia (C) o Disponibilidad (A). El teorema PACELC (Abadi, 2012) formaliza que, en ausencia de partición (E), el compromiso opera entre Latencia (L) y Consistencia (C).
- **Patrón Saga (Richardson, 2018):** Gestión de transacciones en entornos distribuidos mediante secuencias de transacciones locales coordinadas por coreografía o mediación. Los fallos se resuelven mediante *transacciones compensatorias*.
- **Transactional Outbox Pattern:** Garantiza la atomicidad entre la actualización de una base de datos local y la emisión del mensaje correspondiente a un broker, insertando el evento en una tabla de salida dentro de la misma transacción relacional para su lectura posterior mediante Change Data Capture (CDC).

5. Comparación de Arquitecturas de Software

Criterio	Monolito	Capas	Microservicios	Orientada a Eventos	Hexagonal / Clean
Complejidad	Baja	Baja a Media	Alta a Extrema	Alta	Media a Alta
Escalabilidad	Baja (En bloque)	Baja a Media	Muy Alta (Granular)	Extrema (Asíncrona)	Media (Mono-proceso)
Mantenibilidad	Se degrada	Media	Alta por módulo	Media (Flujos difusos)	Máxima (Dominio puro)
Rendimiento	Óptimo (Memoria)	Muy Bueno	Afectado por red	Alto en Throughput	Muy Bueno
Testabilidad	Media	Media (Mocks)	Compleja (Entorno)	Difícil (No determinismo)	Máxima (Sin I/O)
Acoplamiento	Alto	Medio	Bajo (Autónomo)	Mínimo temporal	Muy Bajo (DIP)
Tolerancia Fallo	Nula (Crash total)	Baja	Alta (Aislamiento)	Muy Alta (Buffers)	Depende de infraestructura
Comunicación	Memoria directa	Llamadas / Métodos	REST / gRPC / Eventos	Brokers / Tópicos	Puertos e Interfaces
Gestión Datos	DB única central	DB centralizada	Database-per-service	Event Store / Descentralizada	Aislada tras puertos
Costos Infra.	Mínimos	Bajos	Altos	Altos	Bajos a Medios
Tamaño Equipo	1 a 8 devs	5 a 20 devs	20+ devs (Sub-equipos)	10+ devs	Equipos de cualquier escala
Tipo Proyecto	MVPs, apps simples	CRUD corporativo	Ecosistemas masivos	Streaming, reactivos, IoT	Sistemas de dominio complejo

Tabla 1: Matriz comparativa de estilos y patrones arquitectónicos.

6. Caso Práctico: Plataforma de Comercio Electrónico

6.1. Planteamiento del problema

Se modela un sistema transaccional de comercio electrónico que integra: Catálogo (lectura intensiva, 95 % del tráfico), Carrito (baja latencia en memoria), Procesamiento de Órdenes, Pagos (requerimientos críticos PCI-DSS), Inventario (consistencia estricta) y Notificaciones.

6.2. Evaluación arquitectónica comparada

- **Monolito Modular:** Módulos empaquetados en un único binario con fronteras lógicas estrictas y esquemas de BD separados. Ofrece simplicidad operativa y transacciones locales, pero un fallo en el motor de búsqueda del catálogo compromete la pasarela de pagos.
- **Arquitectura en Capas:** Estructura horizontal genérica con tablas de base de datos interconectadas por claves foráneas. Provoca un acoplamiento crítico entre el modelo de productos y los pedidos, impidiendo el escalamiento independiente.

- **Microservicios Puros (Síncronos):** Despliegue independiente por Bounded Context. Resuelve el escalamiento elástico del Catálogo, pero encadenar peticiones HTTP sincrónicas entre Órdenes, Pagos e Inventario propaga fallos en cascada y eleva la latencia.
- **Orientada a Eventos (EDA):** Flujo asíncrono sobre Kafka. La creación de una orden emite `OrdenCreada`; Pagos procesa el cobro y emite `PagoProcesado`; Inventario reserva stock. Se gestionan fallos mediante el patrón Saga, absorbiendo picos masivos de compra sin colapsar el sistema.

6.3. Selección justificada de la solución técnica

La solución óptima de ingeniería para este escenario consiste en una **Arquitectura Híbrida: Microservicios asíncronos orientados a eventos (EDA) en la infraestructura macro, implementando Arquitectura Hexagonal en el interior de cada servicio individual:**

1. **Protección del Dominio (Hexagonal):** Cada microservicio blindo sus reglas de negocio. Reemplazar pasarelas externas (e.g., de Stripe a Adyen) solo exige modificar un adaptador secundario, sin impacto en la lógica de cobro.
2. **Escalabilidad Asimétrica (Microservicios):** Permite dimensionar de forma elástica 40 réplicas del microservicio de Catálogo sobre bases NoSQL, manteniendo Pagos en 3 instancias protegidas.
3. **Resiliencia y Nivelación de Picos (EDA):** Mediante Kafka y el patrón Saga, la emisión de eventos desacopla temporalmente el procesamiento de compras, evitando caídas durante eventos comerciales de alta concurrencia.
4. **Aislamiento Normativo PCI-DSS:** El servicio de Pagos se despliega en una subred privada segregada con base de datos propia, limitando drásticamente el perímetro de auditoría de seguridad.

7. Guía de Selección y Síntesis Técnica

La selección de una arquitectura no debe responder a tendencias tecnológicas, sino al balance riguroso de restricciones de contexto y compromisos de ingeniería (*trade-offs*):

- **Dominio simple, equipo pequeño o validación de mercado (MVP):** Adoptar un **Monolito Modular** con diseño limpio interno. Minimiza la complejidad operativa y los costes de infraestructura.
- **Sistemas con alta concurrencia transaccional e integración desacoplada:** Utilizar **Arquitecturas Orientadas a Eventos (EDA)**, asumiendo la consistencia eventual a

cambio de máxima reactividad y resiliencia.

- **Organizaciones complejas con múltiples equipos autónomos:** Implementar **Microservicios**, condicionado a la posesión de una alta madurez en orquestación, CI/CD y observabilidad distribuida.
- **Sistemas con reglas de negocio críticas y de alta longevidad:** Estructurar los servicios internamente con **Arquitectura Hexagonal o Clean**, garantizando que el núcleo del negocio sea independiente de frameworks, bases de datos y agentes externos.

Referencias

- [1] Abadi, D. J. (2012). Consistency tradeoffs in modern distributed database system design: CAP is only part of the story. *Computer*, 45(2), 37–42. <https://doi.org/10.1109/MC.2012.33>
- [2] Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.^a ed.). Addison-Wesley Professional.
- [3] Brewer, E. A. (2000). Towards robust distributed systems. *Proceedings of the nineteenth annual ACM symposium on Principles of distributed computing* (PODC '00), 7. <https://doi.org/10.1145/343477.343502>
- [4] Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). *Pattern-oriented software architecture: A system of patterns* (Vol. 1). John Wiley & Sons.
- [5] Chidamber, S. R., & Kemerer, C. F. (1994). A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6), 476–493. <https://doi.org/10.1109/32.295895>
- [6] Cockburn, A. (2005). *Hexagonal architecture: Ports and adapters*. Alistair.Cockburn.us.
- [7] Dijkstra, E. W. (1982). On the role of scientific thought. En *Selected writings on computing: A personal perspective* (pp. 60–66). Springer.
- [8] Erl, T. (2005). *Service-oriented architecture: Concepts, technology, and design*. Prentice Hall.
- [9] Evans, E. (2004). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional.
- [10] Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Tesis doctoral). University of California, Irvine.

- [11] Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley Professional.
- [12] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley Professional.
- [13] IEEE/ISO/IEC. (2011). *ISO/IEC/IEEE 42010:2011: Systems and software engineering — Architecture description*. IEEE. <https://doi.org/10.1109/IEEESTD.2011.6129467>
- [14] Martin, R. C. (2003). *Agile software development, principles, patterns, and practices*. Prentice Hall.
- [15] Martin, R. C. (2017). *Clean architecture: A craftsman’s guide to software structure and design*. Prentice Hall.
- [16] Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.^a ed.). O’Reilly Media.
- [17] Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O’Reilly Media.
- [18] Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.
- [19] Shaw, M., & Garlan, D. (1996). *Software architecture: Perspectives on an emerging discipline*. Prentice Hall.
- [20] Szyperski, C. (2002). *Component software: Beyond object-oriented programming* (2.^a ed.). Addison-Wesley Professional.